

bootsurv: An R Package for Bootstrap Resampling Methods for Survey Data

by Zeinab Mashreghi

Abstract The `bootsurv` package provides a unified framework for applying bootstrap resampling methods to survey data in R. Survey data collected under complex designs, such as stratified simple random sampling and stratified multi-stage cluster sampling, violate the independent and identically distributed assumption underlying the classical bootstrap, requiring design-aware resampling procedures. While several existing R packages implement a small subset of such methods, none offers a comprehensive collection covering important bootstrap methods developed for survey data. The `bootsurv` package fills this gap by implementing three families of bootstrap approaches — pseudo-population, direct, and bootstrap weight methods — covering the most widely used procedures drawn from the survey sampling literature. The package's functions primarily target bootstrap variance estimation and bootstrap statistics in the case of population totals, means, and quartiles, and also provide bootstrap samples and bootstrap replicate weights that can be used to estimate the variance of user-defined estimators. Several survey datasets representing different sampling designs are also included, serving both as worked examples and as general-purpose resources for survey methodology research.

1 Introduction

Survey data from national statistical agencies, such as Statistics Canada, consist of observations representing sampled units along with variables of interest and survey weights. Researchers use this information to calculate statistics that estimate population parameters, such as population totals, means, and quartiles. However, obtaining the variance of a given statistic can be challenging through analytical methods, either when the parameter of interest is complex or when the data are collected under complex survey designs — involving stratification, clustering, and unequal inclusion probabilities — making bootstrap resampling methods a valuable and flexible alternative.

Survey data are generally not independent and identically distributed (i.i.d.); therefore, direct application of the classical bootstrap of [Efron \(1979\)](#) is not appropriate. Modifications are necessary to account for the structural features of complex survey designs. Bootstrap methods designed for stratified simple random sampling have been extensively reviewed and classified into three main categories by [Mashreghi et al. \(2016\)](#): pseudo-population bootstrap methods ([Bickel and Freedman, 1984](#); [Chao and Lo, 1985](#); [Sitter, 1992a](#); [Booth et al., 1994](#); [Chao and Lo, 1994](#)), direct bootstrap methods ([McCarthy and Snowden, 1985](#); [Rao and Wu, 1988](#); [Sitter, 1992b](#)), and bootstrap weight methods ([Rao et al., 1992](#); [Chipperfield and Preston, 2007](#); [Beaumont and Patak, 2012](#); [Bertail and Combris, 1997](#)). Similarly, [Chen et al. \(2022\)](#) examined a range of bootstrap methods for multi-stage cluster sampling, including methods proposed by [Rao et al. \(1992\)](#), [Preston \(2009\)](#), and [Chauvet \(2007\)](#).

To facilitate the application of bootstrap methods for survey data analysis, several R packages are available. The `survey` package ([Lumley, 2004](#)), one of the most widely used tools for complex survey analysis in R, implements bootstrap methods of [Canty and Davison \(1999\)](#), [Rao and Wu \(1988\)](#), and [Preston \(2009\)](#) alongside a broad range of other survey analysis features. The `svrep` package ([Schneider, 2025](#)) extends `survey` by focusing specifically on replicate weight methods, implementing bootstrap weight approaches of [Canty and Davison \(1999\)](#), [Preston \(2009\)](#), [Rao et al. \(1992\)](#), and [Beaumont and Émond \(2022\)](#), as well as the generalized survey bootstrap of [Beaumont and Patak \(2012\)](#). A third package, `surveybootstrap` ([Feehan, 2023](#)), implements the rescaling bootstrap of [Rao and Wu \(1988\)](#) and [Rust and Rao \(1996\)](#). While these packages collectively cover several important methods, each addresses only a subset of the bootstrap procedures available in the survey sampling literature, and none currently implements pseudo-population methods or many

of the other direct bootstrap and bootstrap weight methods. In **survey** and **svrep**, bootstrap variance estimates are obtained by integrating the generated replicate weights into the broader **survey** workflow, while **surveybootstrap** returns bootstrap samples and requires users to define estimators and perform the corresponding variance estimation.

The **bootsurv** package (Mashreghi, 2026) fills this gap by providing a unified and self-contained implementation of all three families of methods identified by Mashreghi et al. (2016) for stratified simple random sampling designs and those examined by Chen et al. (2022) for stratified multi-stage cluster sampling. Bootstrap variance estimates and bootstrap statistics for population totals, means, and quartiles are returned directly, making the package accessible to users regardless of their familiarity with the technical details of bootstrap resampling. The generated bootstrap samples, which include bootstrap weights for the bootstrap weight methods, are also returned, enabling users to extend their analyses to any estimator of interest that can be computed from the sampled units and their survey weights, such as ratios, calibration estimators, and estimates for population subgroups (domains). The package also provides users with bootstrap replicate weights in matrix form, with one row per sampled unit and one column per bootstrap replicate, facilitating the estimation of bootstrap variances for such user-defined estimators. The package is intended for a wide range of users, including survey methodology researchers and survey data analysts in statistical agencies and other applied settings. Several survey datasets representing diverse sampling designs are also included, serving as resources for survey methodology research and education.

The current version focuses on complete-response settings and therefore does not account for the additional variability arising from item nonresponse and imputation. In such settings, variance estimation requires bootstrap procedures that account for the imputation and nonresponse mechanisms (Shao and Sitter, 1996; Mashreghi et al., 2014; Chen et al., 2019, 2021; Mashreghi and Deng, 2023). Extensions to accommodate these settings are potential directions for future work.

This paper is organized as follows. Section 2 introduces the bootstrap methods implemented in **bootsurv**. Section 3 describes the package functions and illustrates their use. Section 4 concludes with a discussion.

2 Bootstrap for survey data

This section introduces the bootstrap methods implemented in **bootsurv**, providing the theoretical background necessary to understand the package functions described in Section 3.

Consider a finite population, U , comprising N distinct units. The parameter of interest, θ , is a function of a survey variable y . To estimate θ , a sample S of size n is selected from U under a sampling design $p(S)$. This sampling design assigns to each unit k in U a first-order inclusion probability π_k and a survey weight $w_k = \pi_k^{-1}$. The estimator of θ , denoted $\hat{\theta}$, is computed from the observed y -values in S and their corresponding survey weights.

For example, a fundamental parameter of interest is the population total $t_y = \sum_{k \in U} y_k$. To estimate t_y , the Horvitz–Thompson estimator proposed by Horvitz and Thompson (1952) is

$$\hat{t}_y = \sum_{k \in S} w_k y_k.$$

To examine the properties of $\hat{\theta}$, a design-based approach is applied, in which the population U remains fixed and the properties of estimators are evaluated over repeated random sampling. The expectation and variance in the design-based framework are defined as follows:

$$E(\hat{\theta}) = \sum_{S \subset U} \hat{\theta}(S) p(S) \quad \text{and} \quad \text{var}(\hat{\theta}) = E[\{\hat{\theta} - E(\hat{\theta})\}^2].$$

An estimator is design-unbiased if $E(\hat{\theta}) = \theta$. The Horvitz–Thompson estimator, $\hat{t}_y$, is design-unbiased for t_y . The design variance of $\hat{t}_y$ is given by

$$\text{var}(\hat{t}_y) = \sum_{k \in U} \sum_{\ell \in U} \Delta_{k\ell} y_k y_\ell, \quad (1)$$

where $\Delta_{k\ell} = (\pi_k \pi_\ell)^{-1}(\pi_{k\ell} - \pi_k \pi_\ell)$ and $\pi_{k\ell}$ denotes the second-order inclusion probability of units k and ℓ . An unbiased estimator of the variance (1) is

$$\hat{\text{var}}(\hat{t}_y) = \sum_{k \in S} \sum_{\ell \in S} \pi_{k\ell}^{-1} \Delta_{k\ell} y_k y_\ell, \quad (2)$$

satisfying $E\{\hat{\text{var}}(\hat{t}_y)\} = \text{var}(\hat{t}_y)$. For example, under simple random sampling without replacement (SRSWOR), $\pi_k = N^{-1}n$ and $\hat{t}_y = N\bar{y}$, where $\bar{y} = n^{-1} \sum_{k \in S} y_k$. Under this design, (2) reduces to the textbook variance estimator,

$$\hat{\text{var}}(\hat{t}_y) = N^2(1 - f)n^{-1}s^2,$$

where $f = N^{-1}n$ is the sampling fraction and $s^2 = (n - 1)^{-1} \sum_{k \in S} (y_k - \bar{y})^2$ is the sample variance of the y -values.

The variance estimator (2) depends on the second-order inclusion probabilities $\pi_{k\ell}$, which can be difficult to obtain under certain unequal probability sampling designs. Furthermore, when the parameter of interest is more complex than a population total, deriving a closed-form variance expression can be challenging or infeasible. In such situations, bootstrap resampling methods provide a practical and flexible approach for variance estimation.

2.1 Bootstrap for stratified simple random sampling

Under stratified simple random sampling without replacement (STSRWOR), the population U of N units is partitioned into L distinct and nonoverlapping strata $U_1, \dots, U_L$, where stratum U_h contains N_h units. Within each stratum, a subsample S_h of size n_h is independently selected using SRSWOR, and the overall sample is $S = \bigcup_{h=1}^L S_h$. The first-order inclusion probability for unit $k \in U_h$ is $\pi_k = N_h^{-1}n_h$, $h = 1, \dots, L$.

To estimate t_y , the Horvitz–Thompson estimator $\hat{t}_y$ is

$$\hat{t}_y = \sum_{k \in S} w_k y_k = \sum_{h=1}^L N_h \bar{y}_h,$$

where $w_k = n_h^{-1}N_h$ for all $k \in S_h$ and $\bar{y}_h$ is the sample mean of the y -values in S_h . The design variance of $\hat{t}_y$ is given by (1), where $\pi_{k\ell} = \{N_h(N_h - 1)\}^{-1}n_h(n_h - 1)$ when both k and ℓ belong to the same stratum U_h , and $\pi_{k\ell} = 0$ otherwise.

In the following sections, three distinct bootstrap approaches under STSRWOR are examined, following the classification of [Mashreghi et al. \(2016\)](#): pseudo-population bootstrap, direct bootstrap, and bootstrap weight methods. This classification provides a useful framework for understanding the similarities and differences among the bootstrap methods proposed for survey data. The corresponding **bootstr** functions implementing these methods are described in Section 3.

• Pseudo-population bootstrap methods

In the pseudo-population approach, the population U is approximated by a pseudo-population constructed by replicating the sample units in S while reflecting the key characteristics of the original sampling design. A bootstrap sample is then drawn from this pseudo-population by applying the same sampling design used for the original sample. Several notable methods within this approach include those developed by [Bickel and Freedman \(1984\)](#), [Chao and Lo \(1985\)](#), [Sitter \(1992a\)](#), [Booth et al. \(1994\)](#), and [Chao and Lo \(1994\)](#).

All these methods follow a unified procedure proposed by Mashreghi et al. (2016), differing only in the method-dependent quantities k_h , n'_h , and U_h^{c*} . For each stratum $h = 1, \dots, L$, a pseudo-stratum U_h^* is constructed by replicating S_h k_h times to obtain a fixed part U_h^f , which is then augmented with a random complement U_h^{c*} drawn from S_h , giving $U_h^* = U_h^f \cup U_h^{c*}$ and $U^* = U_1^* \cup \dots \cup U_L^*$. A bootstrap sample S_h^* of size n'_h is then drawn from U_h^* using SRSWOR, independently across strata, and the bootstrap statistic $\hat{\theta}^*$ is computed from $S^* = S_1^* \cup \dots \cup S_L^*$ using the same estimator as $\hat{\theta}$. The variance estimation involves two nested loops: the inner loop repeats the process of sampling from U^* R_{samp} times to obtain $\hat{\theta}_1^*, \dots, \hat{\theta}_{R_{\text{samp}}}^*$ and to compute

$$\text{var}^* = (R_{\text{samp}} - 1)^{-1} \sum_{r=1}^{R_{\text{samp}}} (\hat{\theta}_r^* - \bar{\theta}^*)^2,$$

where $\bar{\theta}^* = R_{\text{samp}}^{-1} \sum_{r=1}^{R_{\text{samp}}} \hat{\theta}_r^*$, while the outer loop repeats the entire process R_{pop} times to get $\text{var}_1^*, \dots, \text{var}_{R_{\text{pop}}}^*$, yielding the final bootstrap variance estimator

$$\text{var}^* = R_{\text{pop}}^{-1} \sum_{r=1}^{R_{\text{pop}}} \text{var}_r^*. \quad (3)$$

The methods differ in the definition of the replication factor k_h , particularly when N_h/n_h is not an integer. For Booth et al. (1994), Chao and Lo (1994), Bickel and Freedman (1984), and Chao and Lo (1985), the replication factor is given by $k_h = \lfloor N_h/n_h \rfloor$, where $\lfloor \cdot \rfloor$ denotes the integer part. In contrast, the method of Sitter (1992a), known as the bootstrap without-replacement (BWO), modifies the replication factor to $k_h = \lfloor n_h^{-1} N_h \{1 - n_h^{-1} (1 - N_h^{-1} n_h)\} \rfloor$.

The construction of the complement U_h^{c*} differs across methods and follows two distinct approaches. In Booth et al. (1994) and Chao and Lo (1994), U_h^{c*} consists of $N_h - k_h n_h$ units selected from S_h using SRSWOR and simple random sampling with replacement (SRSWR), respectively, fixing the pseudo-stratum size at N_h . In contrast, Bickel and Freedman (1984), Chao and Lo (1985), and Sitter (1992a) construct U_h^{c*} by randomizing between $U_h^{c*} = \emptyset$ with probability q and $U_h^{c*} = S_h$ with probability $1 - q$. The probability q takes a method-specific form, as given in the original references and discussed in Mashreghi et al. (2016).

The bootstrap sample size is $n'_h = n_h$ for Booth et al. (1994), Chao and Lo (1994), Bickel and Freedman (1984), and Chao and Lo (1985). In Sitter (1992a), the bootstrap sample size is modified to $n'_h = n_h - 1_{\{U_h^{c*} = S_h\}}$, so that the sample size is reduced by one when the full subsample is included in the pseudo-stratum.

In all cases, the construction is carried out independently across strata. These methods are implemented in **bootsurv** via the `pseudopop.boot.stsrs` function, described in Section 3.1.

• Direct bootstrap methods

In the direct bootstrap approach, bootstrap samples are generated directly from the original sample S , without constructing a pseudo-population. All methods within this approach build on the classical bootstrap of Efron (1979), which samples with replacement and was originally developed for i.i.d. data, by introducing adjustments to account for the features of survey designs. Within this approach, some methods modify the bootstrap sample size or the observations before resampling with replacement, while one method instead combines independent smaller samples drawn without replacement to form the final bootstrap sample. These methods include those of McCarthy and Snowden (1985), Rao and Wu (1988), and Sitter (1992b), as well as the original method of Efron (1979), all of which are implemented in **bootsurv** via the `direct.boot.stsrs` function, described in Section 3.1.

All direct bootstrap methods follow a unified procedure proposed by Mashreghi et al. (2016), differing in three method-dependent components: the rescaling factor C_h applied to the observations, the bootstrap subsample size n''_h , and the number of subsamples k'_h .

Within each stratum $h = 1, \dots, L$, the observations are first rescaled as

$$y'_k = \bar{y}_h + C_h(y_k - \bar{y}_h), \quad k \in S_h,$$

defining the new sample $S'_h = \{y'_k\}_{k \in S_h}$. A bootstrap sample S_h^* of size $n'_h = k'_h n''_h$ is then obtained by independently drawing k'_h subsamples of size n''_h from S'_h using SRSWOR and then combining them. The overall bootstrap sample is $S^* = S_1^* \cup \dots \cup S_L^*$, and the bootstrap statistic $\hat{\theta}^*$ is computed using the same estimator as $\hat{\theta}$. Repeating this process R times yields $\hat{\theta}_1^*, \dots, \hat{\theta}_R^*$ and the Monte Carlo approximation of the bootstrap variance estimator

$$\text{var}^* = (R-1)^{-1} \sum_{r=1}^R (\hat{\theta}_r^* - \bar{\theta}^*)^2, \quad (4)$$

where $\bar{\theta}^* = R^{-1} \sum_{r=1}^R \hat{\theta}_r^*$.

The simplest methods are those of [Efron \(1979\)](#) and [McCarthy and Snowden \(1985\)](#), both of which apply no rescaling ($C_h = 1$) and draw bootstrap samples using SRSWR — rather than SRSWOR — from the original sample within each stratum ($n''_h = 1$, so that $n'_h = k'_h$). Under [Efron \(1979\)](#), the bootstrap sample size in each stratum equals the original sample size, $n'_h = n_h$. In contrast, to account for finite-population effects, [McCarthy and Snowden \(1985\)](#) set $n'_h = (1 - N_h^{-1}n_h)^{-1}(n_h - 1)$, rounded to the nearest integer.

The method of [Rao and Wu \(1988\)](#) follows the same resampling structure as the preceding methods, using SRSWR ($n''_h = k'_h$) within each stratum with n'_h chosen arbitrarily, but introduces a non-unit rescaling factor

$$C_h = \left\{ \frac{n'_h(1 - N_h^{-1}n_h)}{n_h - 1} \right\}^{1/2}$$

to reflect the without-replacement nature of the original sampling design.

The method of [Sitter \(1992b\)](#), known as the bootstrap mirror-match (BMM), departs more substantially from the preceding methods by allowing k'_h subsamples of a bounded size n''_h , drawn using SRSWOR from the original sample S_h ($C_h = 1$). These k'_h subsamples are then combined to form the final bootstrap sample. Here, $k'_h = \lfloor \{n''_h(1 - N_h^{-1}n_h)\}^{-1}n_h(1 - n_h^{-1}n''_h) \rfloor + I$, where I is a Bernoulli random variable. This randomization is chosen so that the resulting bootstrap variance estimator coincides with the design-based variance estimator (2). The bound on n''_h and the success probability of the Bernoulli distribution are given in [Sitter \(1992b\)](#) and studied in [Mashreghi et al. \(2016\)](#).

• Bootstrap weight methods

Rather than generating bootstrap samples of observations, the bootstrap weight approach produces bootstrap survey weights w_k^* that can be used directly along with the original observed y -values to compute bootstrap statistics $\hat{\theta}^*$ and variance estimates. This concept was introduced by [Rao et al. \(1992\)](#) and is particularly useful when working with public data files released by statistical agencies such as Statistics Canada, which provide datasets with additional columns of bootstrap weights. This allows bootstrap variance estimation without requiring detailed information about the underlying sampling design, thereby preserving confidentiality.

Bootstrap weights take the general form

$$w_k^* = a_k^* w_k,$$

where a_k^* denotes the bootstrap adjustment factor for unit k . All methods considered here, including those of [Rao et al. \(1992\)](#), [Chipperfield and Preston \(2007\)](#), [Beaumont and Patak \(2012\)](#), and [Bertail and Combris \(1997\)](#), construct a_k^* such that $E^*(a_k^*) = 1$, ensuring that the original survey weights are preserved on average under the resampling design.

Under STSRSWOR, the methods of Rao et al. (1992) and Chipperfield and Preston (2007) both define a_k^* as a linear function of the bootstrap frequency n_k^* , denoting the number of times unit k in S_h appears in the bootstrap sample S_h^* of size n_h' . The method of Rao et al. (1992) employs SRSWR with n_h' chosen arbitrarily, subject to conditions ensuring positive bootstrap weights, and defines

$$a_k^* = 1 + \left\{ \frac{n_h'(1 - N_h^{-1}n_h)}{n_h - 1} \right\}^{1/2} \left(\frac{n_h n_k^*}{n_h'} - 1 \right).$$

In contrast, Chipperfield and Preston (2007) use SRSWOR and fix $n_h' = \lfloor n_h/2 \rfloor$, giving

$$a_k^* = 1 + \left\{ \frac{n_h'(1 - N_h^{-1}n_h)}{n_h - n_h'} \right\}^{1/2} \left(\frac{n_h n_k^*}{n_h'} - 1 \right).$$

While both methods rely on a similar linear form in n_k^* , they differ primarily in the resampling scheme and the choice of n_h' .

Bertail and Combris (1997) and Beaumont and Patak (2012) take a different approach by generating the entire adjustment vector $\mathbf{a}_h^* = (a_k^*)_{k \in S_h}$ from a distribution with mean $\mathbf{1}$ and covariance matrix Σ_h , where Σ_h is determined by the sampling design. A practical implementation draws $\mathbf{a}_h^*$ from a multivariate normal distribution $\mathcal{N}(\mathbf{1}, \Sigma_h)$. Since this construction may produce negative adjustments, Beaumont and Patak (2012) propose a modified adjustment $\check{a}_k^* = \tau^{-1}(a_k^* + \tau - 1)$ for a small constant $\tau \geq 1$, with the bootstrap variance estimate multiplied by τ^2 to preserve validity. Full details of Σ_h and the choice of τ are given in Bertail and Combris (1997) and Beaumont and Patak (2012), and further studied in Mashreghi et al. (2016).

The bootstrap variance estimator is approximated using the Monte Carlo variance of the bootstrap statistics (4), as described for the direct bootstrap methods. These methods are implemented in **bootsurv** via the `boot.weights.stsrs` function, described in Section 3.1.

2.2 Bootstrap for stratified two-stage cluster sampling

This section describes bootstrap methods under a stratified two-stage cluster sampling design. Here, stratification is applied at the first stage: the clusters, or primary sampling units (PSUs), are partitioned into strata, and the two-stage sampling procedure is then carried out independently within each stratum. For clarity, the presentation focuses on a single stratum; all methods extend to the stratified case by applying them independently within each stratum and combining the stratum-level bootstrap samples to form the overall bootstrap sample. The **bootsurv** package directly accommodates stratified designs.

Consider a finite population consisting of N nonoverlapping PSUs $U_1, \dots, U_N$, where PSU U_i contains M_i secondary units, $i = 1, \dots, N$. The total number of secondary units in the population is then $M_0 = \sum_{i=1}^N M_i$. To estimate θ , a sample is selected under a two-stage design. In the first stage, a sample S_{1st} of n PSUs is drawn with first- and second-order inclusion probabilities π_i and π_{ij} , respectively. In the second stage, within each selected PSU $i \in S_{1st}$, a sample S_i of m_i elements is drawn independently across PSUs, with conditional inclusion probabilities $\pi_{k|i} = \text{pr}(k \in S_i \mid i \in S_{1st})$ and $\pi_{k\ell|i} = \text{pr}(k, \ell \in S_i \mid i \in S_{1st})$. The overall sample $S = \bigcup_{i \in S_{1st}} S_i$ has size $\sum_{i \in S_{1st}} m_i$.

The population total t_y can be expressed as $t_y = \sum_{i \in U} t_i$, where $t_i = \sum_{k \in U_i} y_{ik}$ is the total for the i th PSU and y_{ik} denotes the y -value for element k in the i th PSU. The Horvitz–Thompson estimator of t_y is

$$\hat{t}_y = \sum_{k \in S} w_k y_k = \sum_{i \in S_{1st}} \sum_{k \in S_i} (\pi_i \pi_{k|i})^{-1} y_{ik},$$

where $w_k = (\pi_i \pi_{k|i})^{-1}$ and $\pi_k = \pi_i \pi_{k|i}$ is the overall inclusion probability of element k . For example, under simple random sampling without replacement at both stages (SR-SWOR/SRSWOR), $\pi_i = N^{-1}n$ and $\pi_{k|i} = M_i^{-1}m_i$.

The variance of $\hat{t}_y$ under a two-stage cluster sampling design can be estimated unbiasedly by

$$\hat{\text{var}}(\hat{t}_y) = \sum_{i \in S_{1st}} \sum_{j \in S_{1st}} \pi_{ij}^{-1} \Delta_{ij} \hat{t}_i \hat{t}_j + \sum_{i \in S_{1st}} \pi_i^{-1} \hat{\text{var}}_i, \quad (5)$$

where $\Delta_{ij} = (\pi_i \pi_j)^{-1}(\pi_{ij} - \pi_i \pi_j)$, $\hat{t}_i = \sum_{k \in S_i} \pi_{k|i}^{-1} y_{ik}$ is the Horvitz–Thompson estimator of t_i ,

$$\hat{\text{var}}_i = \sum_{k \in S_i} \sum_{\ell \in S_i} \pi_{k\ell|i}^{-1} \Delta_{k\ell|i} y_{ik} y_{i\ell}$$

is an unbiased estimator of the within-PSU variance, and $\Delta_{k\ell|i} = (\pi_{k|i} \pi_{\ell|i})^{-1}(\pi_{k\ell|i} - \pi_{k|i} \pi_{\ell|i})$.

Several bootstrap methods have been proposed for variance estimation under two-stage cluster sampling. Under SRSWOR/SRSWOR, the methods considered here include those of Rao and Wu (1988), Rao et al. (1992), Funaoka et al. (2006), Chauvet (2007), and Preston (2009), as well as a modified version of Sitter (1992a) suggested by Chen et al. (2022). The method of Rao et al. (1992) also extends to designs where the first stage uses inclusion probability-proportional-to-size without replacement sampling (IPPSWOR/SRSWOR). The method of Chauvet (2007) is additionally developed for a conditional Poisson sampling design in the first stage (CPS/SRSWOR).

The computation of bootstrap statistics for all methods is described below. For the pseudo-population methods, the bootstrap variance is approximated by (3), following the two nested loops described for the pseudo-population bootstrap methods in Section 2.1. For all other methods, it is computed as the Monte Carlo variance of the bootstrap statistics (4), following the approach outlined for direct and bootstrap weight methods. These methods are implemented in `bootstr` via the `boot.twostage` function, described in Section 3.2.

• Chauvet pseudo-population bootstrap

The method of Chauvet (2007) is a pseudo-population approach developed for high-entropy sampling designs at both stages (e.g., Hájek, 1964; Berger, 1998; Matei and Tillé, 2005; Haziza et al., 2008). In the current implementation, the first stage may use either CPS or SRSWOR, while SRSWOR is used at the second stage. Both designs fall within the high-entropy sampling framework: CPS is a maximum-entropy fixed-size sampling design, while SRSWOR can be viewed as its equal-probability special case.

The pseudo-population U^* is constructed as follows. For each $i \in S_{1st}$, each unit $k \in S_i$ is replicated $\lceil \pi_{k|i}^{-1} \rceil$ times to form a second-stage pseudo-population U_i^* , where $\lceil \cdot \rceil$ denotes the nearest integer. Each pair (S_i, U_i^*) is replicated $\lfloor \pi_i^{-1} \rfloor$ times to create a part of U^* . The remaining part of U^* is completed by selecting further pairs from $\{(S_i, U_i^*)\}_{i \in S_{1st}}$ with probabilities $\pi_i^{-1} - \lfloor \pi_i^{-1} \rfloor$, using SRSWOR when the first stage is SRSWOR and, as suggested by Chauvet (2007), Poisson sampling when the first stage is CPS.

A first-stage bootstrap sample S_{1st}^* is drawn from U^* using the original first-stage sampling design. Within each selected pair $(S_i, U_i^*) \in S_{1st}^*$, a second-stage bootstrap sample S_i^{**} is drawn from U_i^* using the original second-stage design, and $S_i^* = S_i^{**}$ with probability π_i and $S_i^* = S_i$ with probability $1 - \pi_i$. The overall bootstrap sample is $S^* = \bigcup_{i \in S_{1st}^*} S_i^*$, and $\hat{\theta}^*$ is computed on S^* using the same estimator as $\hat{\theta}$.

In the context of high-entropy sampling designs at both stages, Chauvet (2007) demonstrated that the method provides a consistent bootstrap estimator for $\text{var}(\hat{\theta})$.

• Modified Sitter pseudo-population bootstrap

The method of Sitter (1992a) is a pseudo-population approach in which a pseudo-population U^* is constructed by replicating each PSU in S_{1st} k_1 times and each unit within the i th PSU

k_{2i} times, resulting in $N' = nk_1$ PSUs. A bootstrap sample S^* is then obtained by drawing n' PSUs from U^* using SRSWOR and, within each selected PSU, drawing m'_i elements using SRSWOR. The bootstrap statistic $\hat{\theta}^*$ is computed using the same estimator as $\hat{\theta}$.

Sitter (1992a) claimed that the bootstrap variance estimator coincides with (5) under certain conditions on k_1 , n' , k_{2i} , and m'_i . Chen et al. (2022) showed that these conditions require modification for the claim to hold and proposed revised values:

$$k_1 = f_1^{-1}, \quad k_{2i} = f_{2i}^{-1},$$

$$n' = \left\{ 1 + \frac{(1 - f_1)(k_1 n - 1)}{f_1 k_1 (n - 1)} \right\}^{-1} N$$

and

$$m'_i = \left\{ 1 + \frac{f_1(1 - f_{2i})n'(k_{2i}m_i - 1)}{nf_{2i}k_{2i}(m_i - 1)} \right\}^{-1} M_i,$$

where $f_1 = N^{-1}n$ and $f_{2i} = M_i^{-1}m_i$. If any of k_1 , n' , k_{2i} , or m'_i is not an integer, randomization between bracketing integer values is proposed in Sitter (1989).

- **Bernoulli direct bootstrap**

The method of Funaoka et al. (2006) generates a bootstrap sample using Bernoulli random variables. It is a direct bootstrap method, generating bootstrap samples directly from the original sample. At the first stage, an auxiliary sample $\tilde{S}_{1st}$ of size $n' = n - 1$ is drawn from S_{1st} using SRSWR. For each PSU $i \in S_{1st}$, a Bernoulli random variable I_{1i} is generated with probability $p_1 = 1 - \{2(1 - n^{-1})\}^{-1}(1 - f_1)$. If $I_{1i} = 1$, PSU i is retained in the bootstrap sample; if $I_{1i} = 0$, it is replaced by a randomly selected PSU from $\tilde{S}_{1st}$.

At the second stage, for each PSU retained, an auxiliary sample $\tilde{S}_i$ of size $m'_i = m_i - 1$ is drawn from S_i using SRSWR. For each unit $k \in S_i$, a Bernoulli random variable I_{2ik} is generated with probability $p_{2i} = 1 - \{2(1 - m_i^{-1})\}^{-1}p_1\{f_1(1 - f_{2i})\}$. If $I_{2ik} = 1$, unit k is retained in the bootstrap sample S^* ; if $I_{2ik} = 0$, it is replaced by a randomly selected element from $\tilde{S}_i$. The bootstrap statistic $\hat{\theta}^*$ is then computed on S^* using the same estimator as $\hat{\theta}$.

- **Rao–Wu rescaling bootstrap**

The method of Rao and Wu (1988) is based on a rescaling approach applied to the y -values. It is well suited for smooth statistics but may not be appropriate for nonsmooth statistics such as quantiles.

A bootstrap sample is generated in two stages: n PSUs are selected from S_{1st} using SRSWR, and within each selected PSU, a sample of m_i elements is drawn using SRSWR. If a PSU is selected more than once, the within-PSU subsampling is performed independently each time. The y -values are then rescaled by defining

$$\tilde{y}_{ik}^* = \hat{Y} + \gamma_1 \left(\frac{\hat{t}_i^*}{\bar{M}_0} - \hat{Y} \right) + \frac{\gamma_{2i}^*}{\bar{M}_0} (M_i^* y_{ik}^* - \hat{t}_i^*),$$

where y_{ik}^* denotes the y -value of the k th bootstrap element in the i th bootstrap PSU, $\bar{M}_0 = N^{-1}M_0$, $\hat{Y} = \bar{M}_0^{-1}n^{-1} \sum_{i \in S_{1st}} \hat{t}_i$, $\hat{t}_i^* = M_i^* m_i^{*-1} \sum_{k=1}^{m_i^*} y_{ik}^*$,

$$\gamma_1 = \left\{ (n - 1)^{-1}n(1 - f_1) \right\}^{1/2}, \quad \gamma_{2i}^* = \left\{ (m_i^* - 1)^{-1}m_i^* f_1 (1 - M_i^{*-1}m_i^*) \right\}^{1/2},$$

and m_i^* and M_i^* denote the corresponding bootstrap values of m_i and M_i , respectively. The bootstrap statistic $\hat{\theta}^*$ is then computed from the rescaled values $\tilde{y}_{ik}^*$ using the same estimator as $\hat{\theta}$. Rao and Wu (1988) showed that for the population total, the resulting bootstrap variance estimator coincides with (5).

- **Rao–Wu–Yue bootstrap weight**

The method of [Rao et al. \(1992\)](#) is an alternative to [Rao and Wu \(1988\)](#) in which rescaling is applied to the survey weights rather than to the y -values. It yields consistent variance estimators when the first-stage sampling fraction f_1 is negligible, and also extends to IPPSWOR/SRSWOR designs.

Under this method, a bootstrap sample of n' PSUs is selected from S_{1st} using SRSWR, and the bootstrap weights are defined as $w_k^* = a_k^* w_k$, where

$$a_k^* = 1 + \left(\frac{n'}{n-1} \right)^{1/2} \left(\frac{nn_i^*}{n'} - 1 \right),$$

and n_i^* denotes the number of times the i th PSU is selected in the bootstrap sample. The bootstrap statistic $\hat{\theta}^*$ is computed using the same estimator as $\hat{\theta}$, with w_k replaced by w_k^* . Choosing n' such that $0 < n' \leq n-1$ guarantees that the bootstrap weights remain nonnegative.

- **Preston bootstrap weight**

Under the bootstrap weight method of [Preston \(2009\)](#), n' PSUs are selected from S_{1st} using SRSWOR, and within each selected PSU, a sample of m'_i elements is drawn using SRSWOR. The bootstrap weights are defined as $w_k^* = a_k^* \pi_{k|i}^{-1}$, where

$$a_k^* = \left\{ 1 + \lambda_1 \left(\frac{n}{n'} \delta_i - 1 \right) + \lambda_{2i} \left(\frac{n}{n'} \right)^{1/2} \delta_i \left(\frac{m_i}{m'_i} \delta_{ik} - 1 \right) \right\} a_i^{*-1}$$

with $\lambda_1 = \{(n-n')^{-1}n'(1-f_1)\}^{1/2}$, $\lambda_{2i} = \{(m_i - m'_i)^{-1}m'_i f_1(1-f_{2i})\}^{1/2}$, and where δ_i and δ_{ik} are indicator variables for the selection of PSU i and element k within PSU i , respectively, and $a_i^* = 1 + \lambda_1(n'^{-1}n\delta_i - 1)$. The bootstrap statistic $\hat{\theta}^*$ is computed using the same estimator as $\hat{\theta}$, with w_k replaced by w_k^* . An optimal choice, recommended by [Preston \(2009\)](#), is $n' = \lfloor n/2 \rfloor$ and $m'_i = \lfloor m_i/2 \rfloor$, which guarantees nonnegative bootstrap weights.

3 The **bootsurv** package

The **bootsurv** package ([Mashreghi, 2026](#)) is aimed at a broad audience of survey data users, from statistical agency analysts to academic researchers. It implements the bootstrap methods described in Section 2 through four dedicated functions: `pseudopop.boot.stsrs`, `direct.boot.stsrs`, and `boot.weights.stsrs` for pseudo-population, direct, and bootstrap weight methods under STSRSWOR, respectively, and `boot.twostage` for bootstrap methods under stratified cluster sampling. Among the outputs returned by these functions are the vector of bootstrap statistics (`boot.statistic`), the bootstrap variance estimate (`boot.var`), and the bootstrap expectation estimate (`boot.mean`) corresponding to the population totals, means, or quartiles specified by the user. The generated bootstrap samples (`boot.sample`) are also returned. For each bootstrap replicate, `boot.sample` provides detailed information, including the selected units, their bootstrap weights where applicable, and the associated design and resampling information. An additional function, `boot.replicates`, organizes the bootstrap output from the four bootstrap functions by extracting the bootstrap replicate weights in a form that facilitates the computation of user-defined estimators. Together, these outputs allow users to obtain bootstrap variance estimates for estimators corresponding to population parameters beyond the totals, means, and quartiles returned directly by the package. Their use for such user-defined estimators is illustrated in Section 3.3. A complete description of the components returned by each function is provided in the package documentation.

Table 1 summarizes the number of bootstrap methods described in Section 2 that are implemented in **bootsurv** and the existing R packages: **survey**, **svrep**, and **surveybootstrap**. In Table 2, the methods implemented in **bootsurv** are listed by family, sampling design, and resampling scheme.

Table 1: Comparison of the bootstrap methods implemented in `bootsurv` and in existing R packages, by sampling design and method family. Numbers in parentheses indicate the count of bootstrap methods described in Section 2 that are implemented in each package. Partial indicates that only a subset of methods in that family is implemented. Pseudo-pop: pseudo-population; Boot weight: bootstrap weight.

Design	Family	<code>bootsurv</code>	<code>survey</code>	<code>svrep</code>	<code>surveybootstrap</code>
STSRSWOR	Pseudo-pop	✓ (5)	×	×	×
	Direct	✓ (4)	Partial (1)	Partial (1)	Partial (1)
	Boot weight	✓ (4)	Partial (1)	Partial (2)	×
Two-stage cluster	Pseudo-pop	✓ (2)	×	×	×
	Direct	✓ (2)	Partial (1)	Partial (1)	Partial (1)
	Boot weight	✓ (2)	Partial (1)	Partial (2)	×

Table 2: Bootstrap methods implemented in `bootsurv` by family, sampling design, and resampling scheme. Pseudo-pop: pseudo-population; Boot weight: bootstrap weight; WR: with replacement; WOR: without replacement; WR (Bernoulli): with-replacement resampling carried out through Bernoulli selection; distribution: the weight adjustments are drawn from a distribution rather than by resampling. Under the two-stage cluster design, the method of Chauvet (2007) also allows conditional Poisson sampling and that of Rao, Wu, and Yue (1992) also allows IPPS sampling at the first stage.

Design	Family	Bootstrap method	Resampling
STSRSWOR	Pseudo-pop	Bickel.Freedman	WOR
		Chao.Lo.1985	WOR
		Sitter.BWO	WOR
		Chao.Lo.1994	WOR
		Booth.Butler.Hall	WOR
	Direct	Efron	WR
		McCarthy.Snowden	WR
		Sitter.BMM	WOR
		Rao.Wu	WR
	Boot weight	Rao.Wu.Yue	WR
		Chipperfield.Preston	WOR
		Bertail.Combris	distribution
		Beaumont.Patak	distribution
Two-stage cluster	Pseudo-pop	Chauvet	WOR
		Modified.Sitter	WOR
	Direct	Funaoka.etal	WR (Bernoulli)
		Rao.Wu	WR
	Boot weight	Rao.Wu.Yue	WR
		Preston	WOR

The package functions are illustrated throughout this section using the datasets included in [bootsurv](#). The populations `data_pop` and `data_pop_st` correspond to unstratified and stratified designs, with samples `data_samp_srs` and `data_samp_stsrs` drawn under SRSWOR and STSRSWOR, respectively. The populations `data_pop_clust` and `data_pop_stclust` correspond to cluster and stratified cluster designs, with samples `data_samp_clust` and `data_samp_stclust` drawn under two-stage and stratified two-stage cluster sampling, respectively. Beyond their use as worked examples, these datasets can serve as general-purpose resources for survey methodology research and education. All datasets can be loaded as follows:

```
library(bootsurv)
data(data_pop, package = "bootsurv")
data(data_samp_srs, package = "bootsurv")
data(data_pop_st, package = "bootsurv")
data(data_samp_stsrs, package = "bootsurv")
data(data_pop_clust, package = "bootsurv")
data(data_samp_clust, package = "bootsurv")
data(data_pop_stclust, package = "bootsurv")
data(data_samp_stclust, package = "bootsurv")
```

3.1 [bootsurv](#) functions for stratified simple random sampling

This section describes the three [bootsurv](#) functions implementing the bootstrap methods under STSRSWOR described in Section 2.1 and illustrates their use on the included datasets.

The function `pseudopop.boot.stsrs` implements the pseudo-population bootstrap methods and is called as

```
pseudopop.boot.stsrs(data, population.size, R.pop, R.samp, study.variable,
                     stratum = NULL, parameter, bootstrap.method)
```

Here, `data` is a vector, matrix, or data frame containing the sample observations or sample information. When `data` is provided as a matrix or data frame, the column containing the study variable values (y -values) is identified by the argument `study.variable` (default `"study.variable"`), and, under STSRSWOR, the column identifying the strata is given by the argument `stratum` (default `NULL`, corresponding to an unstratified SRSWOR design). The argument `population.size` specifies the population size N for an unstratified design or the vector of strata population sizes $N_1, \dots, N_L$ for a stratified design.

The arguments `R.pop` and `R.samp` denote R_{pop} and R_{samp} , respectively. The argument `parameter` specifies the target parameter and accepts `"total"` (default), `"mean"`, `"quartile.25"`, `"quartile.50"` (or `"median"`), or `"quartile.75"`. The Horvitz–Thompson estimator is used for the total and mean, while the estimator of [Särndal et al. \(1992, Chapter 5\)](#) is applied for quartiles. The argument `bootstrap.method` specifies the bootstrap method and accepts `"Bickel.Freedman"` ([Bickel and Freedman, 1984](#)), `"Chao.Lo.1985"` ([Chao and Lo, 1985](#)), `"Sitter.BW0"` ([Sitter, 1992a](#)), `"Chao.Lo.1994"` ([Chao and Lo, 1994](#)), and `"Booth.Butler.Hall"` ([Booth et al., 1994](#)), with `"Booth.Butler.Hall"` as the default.

The function `direct.boot.stsrs` implements the direct bootstrap methods and is called as

```
direct.boot.stsrs(data, population.size, R, study.variable, stratum = NULL,
                  parameter, bootstrap.method, boot.sample.size = NULL)
```

The arguments `data`, `population.size`, `study.variable`, `stratum`, and `parameter` are as defined in the `pseudopop.boot.stsrs` function.

The argument `R` denotes the number of bootstrap replicates in (4). The argument `bootstrap.method` accepts `"Efron"` ([Efron, 1979](#)), `"McCarthy.Snowden"` ([McCarthy and](#)

Snowden, 1985), "Sitter.BMM" (Sitter, 1992b), and "Rao.Wu" (Rao and Wu, 1988), with "Rao.Wu" as the default. The argument `boot.sample.size` applies only for the method of Rao and Wu (1988), where a vector of bootstrap sample sizes n'_h for each stratum may be specified. If not provided, n'_h defaults to $n_h - 3$ for $h = 1, \dots, L$.

The function `boot.weights.stsrs` implements the bootstrap weight methods and is called as

```
boot.weights.stsrs(data, population.size, R, study.variable, stratum = NULL,
                  parameter, bootstrap.method, boot.sample.size = NULL,
                  distribution.adjust = NULL, epsilon = NULL)
```

The arguments `data`, `population.size`, `R`, `study.variable`, `stratum`, and `parameter` are as defined in the previous functions. The argument `bootstrap.method` accepts "Bertail.Combris" (Bertail and Combris, 1997), "Chipperfield.Preston" (Chipperfield and Preston, 2007), "Beaumont.Patak" (Beaumont and Patak, 2012), and "Rao.Wu.Yue" (Rao et al., 1992), with "Rao.Wu.Yue" as the default.

For the method of Rao et al. (1992), a vector of bootstrap sample sizes within strata, `boot.sample.size`, is required; if not provided, n'_h defaults to $n_h - 1$ for $h = 1, \dots, L$.

The arguments `distribution.adjust` and `epsilon` apply to the methods of Bertail and Combris (1997) and Beaumont and Patak (2012). When `epsilon = NULL`, `distribution.adjust` specifies the distribution used to generate the bootstrap weight adjustments, accepting "Normal", "Lognormal", "Exponential", or "Uniform". When `distribution.adjust = NULL`, the argument `epsilon` controls the application of the distribution described in Beaumont and Patak (2012).

The three functions are illustrated using `data_samp_srs` under SRSWOR and `data_samp_stsrs` under STSRWOR with $L = 5$ strata. For `data_samp_srs`, the method of Booth et al. (1994) is applied via `pseudopop.boot.stsrs`, the rescaling method of Rao and Wu (1988) via `direct.boot.stsrs`, and the bootstrap weight method of Chipperfield and Preston (2007) via `boot.weights.stsrs`, with the population total, mean, and median as the target parameters, respectively. For `data_samp_stsrs`, the bootstrap weight method of Rao et al. (1992) is applied via `boot.weights.stsrs` to estimate $\text{var}(\hat{f}_y)$. Additionally, some of the $R = 500$ bootstrap statistics generated by the method of Rao and Wu (1988) and some of the bootstrap weights generated by the method of Chipperfield and Preston (2007) are presented.

```
# SRSWOR - Booth-Butler-Hall for the population total
population.size <- nrow(data_pop)
R.pop <- 5
R.samp <- 100
boot.BBH <- pseudopop.boot.stsrs(data_samp_srs, population.size, R.pop, R.samp)
boot.BBH$boot.var

#> [1] 2742881

# SRSWOR - Rao-Wu for the population mean
R <- 500
boot.RW <- direct.boot.stsrs(data_samp_srs, population.size, R,
                           parameter = "mean")
boot.RW$boot.var

#> [1] 0.08524448

head(boot.RW$boot.statistic)

#> [1] 49.83364 50.19668 50.15833 49.10216 49.25829 50.17160
```

```
# SRSWOR - Chipperfield-Preston for the population median
boot.CP <- boot.weights.stsrs(data_samp_srs, population.size, R,
                             parameter = "median",
                             bootstrap.method = "Chipperfield.Preston")

boot.CP$boot.var

#> [1] 0.2081661

head(boot.CP$boot.sample[[1]]$bootstrap.weight)

#> [1] 5.9405351 5.9405351 5.9405351 5.9405351 5.9405351 0.5459514

# STSRSWOR - Rao-Wu-Yue for the population total
population.size.st <- as.vector(table(data_pop_st$stratum))
boot.RWY.st <- boot.weights.stsrs(data_samp_stsrs, population.size.st,
                                  R = 1000, stratum = "stratum")

boot.RWY.st$boot.var

#> [1] 9711264
```

To further validate the results, the bootstrap variance estimates are compared with those obtained using the Rao–Wu–Yue–Beaumont method implemented in [svrep](#), for different population parameters under SRSWOR and STSRSWOR. These methods are expected to yield comparable, but not identical, results, as they rely on Monte Carlo approximation.

```
library(survey)
library(svrep)

# SRSWOR - Rao-Wu-Yue-Beaumont
svrep.design.srs <- svydesign(data = data_samp_srs, ids = ~ 1,
                             fpc = ~ rep(population.size, nrow(data_samp_srs)))
svrep.boot.srs <- as_bootstrap_design(design = svrep.design.srs,
                                     type = "Rao-Wu-Yue-Beaumont",
                                     replicates = R)

# Bootstrap variance estimate for the population total
SE(svytotal(~ study.variable, design = svrep.boot.srs))^2

#> [1] 3053405

# Bootstrap variance estimate for the population mean
SE(svymean(~ study.variable, design = svrep.boot.srs))^2

#> [1] 0.0848168

# Bootstrap variance estimate for the population median
SE(svyquantile(~ study.variable, design = svrep.boot.srs, quantiles = 0.5))^2

#> study.variable
#> 0.210669

# STSRSWOR - Rao-Wu-Yue-Beaumont for the population total
svrep.design.stsrs <- svydesign(data = data_samp_stsrs, ids = ~ 1,
                               strata = ~ stratum,
                               fpc = ~ rep(population.size.st,
                                             table(data_samp_stsrs$stratum)))
```

```
svrep.boot.stsrs <- as_bootstrap_design(design = svrep.design.stsrs,
                                       type = "Rao-Wu-Yue-Beaumont",
                                       replicates = 1000)

SE(svytotal(~ study.variable, design = svrep.boot.stsrs))^2

#> [1] 9195383
```

3.2 `bootsurv` function for cluster sampling

The function `boot.twostage` implements the bootstrap methods described in Section 2.2 and is called as

```
boot.twostage(data, no.cluster, cluster.size, R, study.variable,
              stratum = NULL, cluster, Pi1 = NULL, parameter,
              bootstrap.method, survey.design, population.size = NULL,
              boot.sample.size = NULL)
```

The argument `data` is defined as in the previous functions. The study variable and strata columns are specified by the arguments `study.variable` and `stratum`, respectively; in addition, the cluster (PSU) column is identified by `cluster`, and, when an IPPS or CPS design is used at the first stage, the column of first-stage inclusion probabilities is specified by `Pi1`. The argument `no.cluster` is a vector giving the total number of clusters within each stratum, and `cluster.size` is a vector containing the population size of each selected cluster, with length equal to the total number of selected clusters across all strata. The arguments `R` and `parameter` are defined as in the previous functions, with the exception that, when the method of [Chauvet \(2007\)](#) is used, `R` must be specified as a vector of two values ($R_{\text{pop}}, R_{\text{samp}}$) representing the number of bootstrap pseudo-populations and the number of bootstrap samples drawn from each, respectively.

The argument `survey.design` specifies the first-stage sampling design and accepts "SRSWOR" (default), "CPS" (only for the Chauvet pseudo-population method), or "IPPS" (only for the Rao–Wu–Yue bootstrap weight method). The argument `bootstrap.method` specifies the bootstrap procedure and accepts "Rao.Wu" ([Rao and Wu, 1988](#)), "Rao.Wu.Yue" ([Rao et al., 1992](#)), "Funaoka.etal" ([Funaoka et al., 2006](#)), "Chauvet" ([Chauvet, 2007](#)), "Preston" ([Preston, 2009](#)), and "Modified.Sitter" ([Chen et al., 2022](#)), with "Rao.Wu.Yue" as the default.

The argument `population.size` is the population size for an unstratified design or the vector of strata population sizes required for the "Rao.Wu" method and when estimating the population mean. The argument `boot.sample.size` specifies bootstrap sample sizes within strata, required only for the "Rao.Wu.Yue" method; if not provided, n'_h defaults to $n_h - 1$ for $h = 1, \dots, L$.

The function is illustrated using `data_samp_clust` under SRSWOR/SRSWOR and `data_samp_stclust` under stratified SRSWOR/SRSWOR with $L = 3$. The method of [Funaoka et al. \(2006\)](#) is first applied to estimate $\text{var}(\hat{f}_y)$, and the method of [Rao et al. \(1992\)](#) to return the bootstrap variance of the population median and some of the $R = 500$ generated bootstrap estimates, both using `data_samp_clust`. The method of [Preston \(2009\)](#) is then applied to `data_samp_stclust` to estimate $\text{var}(\hat{f}_y)$ under stratified two-stage cluster sampling.

```
# SRSWOR/SRSWOR - Funaoka et al. for the population total
no.cluster <- length(unique(data_pop_clust$cluster))
cluster.size <- table(data_pop_clust$cluster)[unique(data_samp_clust$cluster)]
R <- 500
boot.Fu <- boot.twostage(data_samp_clust, no.cluster, cluster.size, R,
                        bootstrap.method = "Funaoka.etal")
boot.Fu$boot.var
```

```

#> [1] 5791315

# SRSWOR/SRSWOR - Rao-Wu-Yue for the population median
boot.RWY <- boot.twostage(data_samp_clust, no.cluster, cluster.size, R,
                          parameter = "median")
boot.RWY$boot.var

#> [1] 0.01148908

head(boot.RWY$boot.statistic)

#> [1] 10.196255  9.940355 10.210666  9.922516 10.322815 10.210666

# Stratified SRSWOR/SRSWOR - Preston for the population total
no.cluster.stclust <- c(100, 125, 65)
cluster.size.pop.st <- aggregate(data_pop_stclust$cluster,
                                 by = list(data_pop_stclust$stratum), table)[[2]]
L <- length(unique(data_samp_stclust$stratum))
cluster.size.st <- NULL
for (h in 1:L) {
  cluster.size.st <- c(cluster.size.st,
                      cluster.size.pop.st[[h]]
                      [unique(data_samp_stclust$cluster
                              [data_samp_stclust$stratum == h])])
}
boot.Pr.st <- boot.twostage(data_samp_stclust, no.cluster.stclust,
                           cluster.size.st, R = 1000, stratum = "stratum",
                           bootstrap.method = "Preston")
boot.Pr.st$boot.var

#> [1] 8264138

```

To further validate the results, the bootstrap variance estimates obtained via `boot.twostage` are compared with those obtained using the Preston bootstrap method implemented in the [survey](#) package, for the population total and median under SRSWOR/SRSWOR, and for the population total under stratified SRSWOR/SRSWOR. The methods are expected to yield comparable but not identical results, as they differ in their resampling schemes and both rely on Monte Carlo approximation.

```

library(survey)

# SRSWOR/SRSWOR - Preston
fpc2 <- as.numeric(cluster.size[as.character(data_samp_clust$cluster)])
survey.design.clust <- svydesign(data = data_samp_clust,
                               ids = ~ cluster + id,
                               fpc = ~ rep(no.cluster, nrow(data_samp_clust)) + fpc2)
survey.boot.clust <- as.svrepdesign(design = survey.design.clust,
                                 type = "mrbootstrap",
                                 replicates = R)

# Bootstrap variance estimate for the population total
SE(svytotal(~ study.variable, design = survey.boot.clust))^2

#> [1] 5729084

# Bootstrap variance estimate for the population median
SE(svyquantile(~ study.variable, design = survey.boot.clust, quantiles = 0.5))^2

```

```
#> study.variable
#>      0.01066904

# Stratified SRSWOR/SRSWOR - Preston for the population total
fpc2.st <- as.numeric(cluster.size.st[as.character(data_samp_stclust$cluster)])
survey.design.stclust <- svydesign(data = data_samp_stclust,
                                ids = ~ cluster + id,
                                strata = ~ stratum,
                                fpc = ~ rep(no.cluster.stclust,
                                             table(data_samp_stclust$stratum)) + fpc2.st,
                                nest = TRUE)
survey.boot.stclust <- as.svrepdesign(design = survey.design.stclust,
                                   type = "mrbootstrap",
                                   replicates = 1000)

SE(svytotal(~ study.variable, design = survey.boot.stclust))^2

#> [1] 8834462
```

Although `boot.twostage` is presented for two-stage cluster sampling, it can also be applied to one-stage cluster sampling as a special case in which all secondary units within each selected PSU are included in the sample. In this case, the second-stage sample coincides with the PSU population, so that $m_i = M_i$.

3.3 User-defined estimators and bootstrap replicates

The bootstrap samples returned by each bootstrap function in the package (`boot.sample`) provide, for each replicate, the selected units, their bootstrap weights where applicable, and the associated design and resampling information. This information can be used to estimate the variance of estimators beyond the totals, means, and quartiles produced directly by the package. To facilitate such analyses, the function `boot.replicates` converts the output of the four bootstrap functions into a matrix of bootstrap replicate weights, with one row for each unit in the original sample and one column for each bootstrap replicate. This applies to every method except that of [Rao and Wu \(1988\)](#) under a two-stage design, which is discussed at the end of this subsection. Such a matrix format is commonly used for survey data that include replicate weights. With this matrix, the bootstrap variance of an estimator is obtained by computing the estimator for each set of replicate weights and applying the corresponding bootstrap variance formula to the resulting replicate estimates.

The following examples illustrate this procedure for bootstrap weight, direct, and pseudo-population methods and verify that the resulting variance estimates reproduce those returned by the corresponding bootstrap functions. For the pseudo-population method, the variance is formed over the two nested levels of the pseudo-population procedure.

```
# SRSWOR - Rao-Wu-Yue for the population total
R <- 500
boot.RWY <- boot.weights.stsrs(data_samp_srs, population.size, R)
rep.weights <- boot.replicates(boot.RWY)
dim(rep.weights)

#> [1] 1850  500

# The replicate weights from boot.replicates reproduce
# the boot.weights.stsrs bootstrap variance estimate
y <- data_samp_srs$study.variable
c(var(apply(rep.weights, 2, function(w) sum(w * y))), boot.RWY$boot.var)
```

```
#> [1] 3104454 3104454

# SRSWOR - Sitter.BMM for the population total
boot.BMM <- direct.boot.stsrs(data_samp_srs, population.size, R,
                             bootstrap.method = "Sitter.BMM")

# The replicate weights from boot.replicates reproduce
# the direct.boot.stsrs bootstrap variance estimate
c(var(apply(boot.replicates(boot.BMM), 2, function(w) sum(w * y))),
  boot.BMM$boot.var)

#> [1] 3298361 3298361

# SRSWOR - Pseudo-population method of Booth-Butler-Hall
# for the population total
# The variance is formed over the two nested levels of the pseudo-pop procedure
boot.BBH <- pseudopop.boot.stsrs(data_samp_srs, population.size,
                                R.pop = 5, R.samp = 100)

# The replicate weights from boot.replicates reproduce
# the pseudopop.boot.stsrs bootstrap variance estimate
totals <- apply(boot.replicates(boot.BBH), 2, function(w) sum(w * y))
pop <- rep(1:5, each = 100)
c(mean(tapply(totals, pop, var)), boot.BBH$boot.var)

#> [1] 2962975 2962975
```

Because the rows of the matrix follow the order of the original sample, the replicate weights can be applied directly to any study variable or function of the sampled data. The next example uses the same set of replicate weights generated by the [Rao et al. \(1992\)](#) method to obtain bootstrap variance estimates for a domain total, a ratio, two study variables simultaneously, and a calibration estimator.

```
# SRSWOR - Rao-Wu-Yue
# Total over a subgroup (domain): units with study variable above its median
domain <- y > median(y)
var(apply(rep.weights, 2, function(w) sum(w[domain] * y[domain])))

#> [1] 12944956

# Ratio of the domain total to the overall population total
var(apply(rep.weights, 2, function(w) sum(w[domain] * y[domain]) / sum(w * y)))

#> [1] 8.242414e-05

# Bootstrap variances for multiple study variables simultaneously
# x-values are generated for illustration and linked to the sample
# through the unit id
set.seed(1)
x.pop <- 2 + 0.5 * data_pop$study.variable + rnorm(nrow(data_pop), 0, 5)
x <- x.pop[match(data_samp_srs$id, data_pop$id)]
apply(crossprod(rep.weights, cbind(y = y, x = x)), 2, var)

#>      y      x
#> 3104454 1059073
```

```
# Calibration estimator of the total of y, calibrated to the known total of x
t.x <- sum(x.pop)
var(apply(rep.weights, 2, function(w) t.x * sum(w * y) / sum(w * x)))

#> [1] 1162420
```

The method of [Rao and Wu \(1988\)](#) rescales the study variable to account for finite-population and survey design features. Under SRSWOR, this rescaling can still be expressed through replicate weights, so `boot.replicates` applies directly. Under STSRSWOR, the bootstrap variance estimate is instead reproduced from `boot.sample` and the rescaling factor (`rescale.factor`) by rescaling the study values within each stratum. Under a two-stage cluster sampling design, this rescaling also involves the cluster and population sizes, which are not included in the returned bootstrap information; therefore, the Rao–Wu bootstrap can be reproduced for the original study variable, but not directly for additional study variables. The other bootstrap methods for two-stage cluster sampling are handled by `boot.replicates` as above.

```
# SRSWOR - Rao-Wu for the population total
boot.RW <- direct.boot.stsrs(data_samp_srs, population.size, R,
                             bootstrap.method = "Rao.Wu")

# The replicate weights from boot.replicates reproduce
# the Rao-Wu bootstrap variance estimate
c(var(apply(boot.replicates(boot.RW), 2, function(w) sum(w * y))),
  boot.RW$boot.var)

#> [1] 3238211 3238211

# STSRSWOR - Rao-Wu for the population total
# Rescale the study variable values within each stratum using
# boot.sample and rescale.factor
population.size.st <- as.vector(table(data_pop_st$stratum))
boot.RW.st <- direct.boot.stsrs(data_samp_stsrs, population.size.st, R,
                                stratum = "stratum",
                                bootstrap.method = "Rao.Wu")

y.st <- data_samp_stsrs$study.variable
C <- boot.RW.st$rescale.factor
strata <- sort(unique(data_samp_stsrs$stratum))
ybar <- tapply(y.st, data_samp_stsrs$stratum, mean)
total.RW.st <- sapply(boot.RW.st$boot.sample, function(boot.rep) {
  h <- match(boot.rep$stratum, strata)
  sum(boot.rep$bootstrap.weight *
      (ybar[h] + C[h] * (y.st[boot.rep$row.id] - ybar[h])))
})
# The bootstrap variance estimate reproduced from boot.sample
# is the same as the Rao-Wu bootstrap variance estimate
c(var(total.RW.st), boot.RW.st$boot.var)

#> [1] 9615924 9615924
```

4 Summary and discussion

The `bootsurv` package provides a unified framework for applying bootstrap resampling methods to survey data collected under complex sampling designs. It enables bootstrap variance estimation and bootstrap statistics for population totals, means, and quartiles, while also returning the generated bootstrap samples and, where applicable, bootstrap

weights, allowing users to extend their analyses to additional estimators of interest. In particular, the additional function `boot.replicates` organizes the output of the bootstrap functions into bootstrap replicate weights, facilitating variance estimation for user-defined estimators. These capabilities allow users to apply the implemented methods beyond the totals, means, and quartiles returned directly by the package, including domain totals, ratios, and calibration estimators.

The package implements bootstrap procedures for stratified simple random sampling without replacement and stratified two-stage cluster sampling, covering the most widely used methods across three families: pseudo-population, direct, and bootstrap weight approaches. The availability of multiple procedures within a single framework allows users to select methods appropriate for their sampling design and parameter of interest.

The choice among these methods depends mainly on the parameter of interest and the sampling design, as summarized in Table 2. Under the designs considered, the pseudo-population and direct methods provide consistent variance estimation for smooth functions of totals, with the exception of the classical bootstrap of Efron (1979), which does not incorporate the finite-population correction. The bootstrap weight method of Rao et al. (1992) incorporates the finite-population correction under stratified simple random sampling, where it is consistent for a nonnegligible sampling fraction; under two-stage cluster sampling, it is consistent when the first-stage sampling fraction is negligible. With the single exception of the rescaling method of Rao and Wu (1988), which rescales the study-variable values and is therefore intended for smooth functions of totals rather than quantiles, the other implemented methods can also be used to estimate the variance of quantiles. These methods are classified and compared by Mashreghi et al. (2016) for stratified simple random sampling and by Chen et al. (2022) for stratified two-stage cluster sampling.

While existing R packages such as `survey` (Lumley, 2004), `svrep` (Schneider, 2025), and `surveybootstrap` (Feehan, 2023) implement a limited selection of bootstrap procedures (Table 1), `bootsurv` offers a more comprehensive collection within a unified interface.

The datasets included in `bootsurv` cover a range of sampling designs and serve both as worked examples throughout the package documentation and as general-purpose resources for survey methodology research and education.

The current version of `bootsurv` focuses on complete-response settings. Future development could broaden the applicability of the package to additional survey settings, including those involving item nonresponse and imputed data. Overall, `bootsurv` provides a comprehensive resource for bootstrap-based inference under complex survey designs.

Acknowledgments

This research was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).

References

- J.-F. Beaumont and N. Émond. A bootstrap variance estimation method for multistage sampling and two-phase sampling when Poisson sampling is used at the second phase. *Stats*, 5(2):339–357, 2022. doi: 10.3390/stats5020019. [p1]
- J.-F. Beaumont and Z. Patak. On the generalized bootstrap for sample surveys with special attention to Poisson sampling. *International Statistical Review*, 80(1):127–148, 2012. doi: 10.1111/j.1751-5823.2011.00166.x. [p1, 5, 6, 12]
- Y. G. Berger. Rate of convergence for asymptotic variance of the Horvitz–Thompson estimator. *Journal of Statistical Planning and Inference*, 74(1):149–168, 1998. doi: 10.1016/S0378-3758(98)00107-4. [p7]

- P. Bertail and P. Combris. Bootstrap généralisé d'un sondage. *Annales d'Économie et de Statistique*, (46):49–83, 1997. doi: 10.2307/20076068. [p1, 5, 6, 12]
- P. J. Bickel and D. A. Freedman. Asymptotic normality and the bootstrap in stratified sampling. *The Annals of Statistics*, 12(2):470–482, 1984. doi: 10.1214/aos/1176346500. [p1, 3, 4, 11]
- J. G. Booth, R. W. Butler, and P. Hall. Bootstrap methods for finite populations. *Journal of the American Statistical Association*, 89(428):1282–1289, 1994. doi: 10.1080/01621459.1994.10476868. [p1, 3, 4, 11, 12]
- A. J. Canty and A. C. Davison. Resampling-based variance estimation for labour force surveys. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 48(3):379–391, 1999. doi: 10.1111/1467-9884.00196. [p1]
- M.-T. Chao and S.-H. Lo. A bootstrap method for finite population. *Sankhyā: The Indian Journal of Statistics, Series A*, 47(3):399–405, 1985. [p1, 3, 4, 11]
- M.-T. Chao and S.-H. Lo. Maximum likelihood summary and the bootstrap method in structured finite populations. *Statistica Sinica*, 4(2):389–406, 1994. [p1, 3, 4, 11]
- G. Chauvet. *Méthodes de bootstrap en population finie*. PhD thesis, École Nationale de la Statistique et de l'Analyse de l'Information, Bruz, France, 2007. [p1, 7, 14]
- S. Chen, D. Haziza, C. Léger, and Z. Mashreghi. Pseudo-population bootstrap methods for imputed survey data. *Biometrika*, 106(2):369–384, 2019. doi: 10.1093/biomet/asz001. [p2]
- S. Chen, D. Haziza, and Z. Mashreghi. Multiply robust bootstrap variance estimation in the presence of singly imputed survey data. *Journal of Survey Statistics and Methodology*, 9(4): 810–832, 2021. doi: 10.1093/jssam/smaa004. [p2]
- S. Chen, D. Haziza, and Z. Mashreghi. A comparison of existing bootstrap algorithms for multi-stage sampling designs. *Stats*, 5(2):521–537, 2022. doi: 10.3390/stats5020031. [p1, 2, 7, 8, 14, 19]
- J. Chipperfield and J. Preston. Efficient bootstrap for business surveys. *Survey Methodology*, 33(2):167–172, 2007. [p1, 5, 6, 12]
- B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552. [p1, 4, 5, 11, 19]
- D. M. Feehan. *surveybootstrap: Bootstrap with Survey Data*, 2023. URL <https://CRAN.R-project.org/package=surveybootstrap>. R package version 0.0.3. [p1, 19]
- F. Funaoka, H. Saigo, R. R. Sitter, and T. Toida. Bernoulli bootstrap for stratified multistage sampling. *Survey Methodology*, 32(2):151–156, 2006. [p7, 8, 14]
- J. Hájek. Asymptotic theory of rejective sampling with varying probabilities from a finite population. *The Annals of Mathematical Statistics*, 35(4):1491–1523, 1964. doi: 10.1214/aoms/1177700375. [p7]
- D. Haziza, F. Mecatti, and J. N. K. Rao. Evaluation of some approximate variance estimators under the Rao–Sampford unequal probability sampling design. *Metron*, 66(1):91–108, 2008. [p7]
- D. G. Horvitz and D. J. Thompson. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association*, 47(260):663–685, 1952. doi: 10.1080/01621459.1952.10483446. [p2]
- T. Lumley. Analysis of complex survey samples. *Journal of Statistical Software*, 9(8):1–19, 2004. doi: 10.18637/jss.v009.i08. [p1, 19]

- Z. Mashreghi. *bootsurv: Bootstrap Methods for Complete Survey Data*, 2026. URL <https://CRAN.R-project.org/package=bootsurv>. R package version 0.1.0. [p2, 9]
- Z. Mashreghi and H. Deng. A rescaling bootstrap approach for imputed survey data. *Journal of Survey Statistics and Methodology*, 11(1):234–259, 2023. doi: 10.1093/jssam/smab047. [p2]
- Z. Mashreghi, C. Léger, and D. Haziza. Bootstrap methods for imputed data from regression, ratio and hot-deck imputation. *The Canadian Journal of Statistics*, 42(1):142–167, 2014. doi: 10.1002/cjs.11206. [p2]
- Z. Mashreghi, D. Haziza, and C. Léger. A survey of bootstrap methods in finite population sampling. *Statistics Surveys*, 10:1–52, 2016. doi: 10.1214/16-SS113. [p1, 2, 3, 4, 5, 6, 19]
- A. Matei and Y. Tillé. Evaluation of variance approximations and estimators in maximum entropy sampling with unequal probability and fixed sample size. *Journal of Official Statistics*, 21(4):543–570, 2005. [p7]
- P. J. McCarthy and C. B. Snowden. The bootstrap and finite population sampling. Vital and Health Statistics Series 2, No. 95, DHHS Publication No. (PHS) 85-1369, U.S. Government Printing Office, Washington, DC, 1985. [p1, 4, 5, 11]
- J. Preston. Rescaled bootstrap for stratified multistage sampling. *Survey Methodology*, 35(2): 227–234, 2009. [p1, 7, 9, 14]
- J. N. K. Rao and C. F. J. Wu. Resampling inference with complex survey data. *Journal of the American Statistical Association*, 83(401):231–241, 1988. doi: 10.1080/01621459.1988.10478591. [p1, 4, 5, 7, 8, 9, 12, 14, 16, 18, 19]
- J. N. K. Rao, C. F. J. Wu, and K. Yue. Some recent work on resampling methods for complex surveys. *Survey Methodology*, 18(2):209–217, 1992. [p1, 5, 6, 7, 9, 12, 14, 17, 19]
- K. F. Rust and J. N. K. Rao. Variance estimation for complex surveys using replication techniques. *Statistical Methods in Medical Research*, 5(3):283–310, 1996. doi: 10.1177/096228029600500305. [p1]
- C.-E. Särndal, B. Swensson, and J. Wretman. *Model Assisted Survey Sampling*. Springer Series in Statistics. Springer-Verlag, New York, 1992. doi: 10.1007/978-1-4612-4378-6. [p11]
- B. Schneider. *svrep: Tools for Creating, Updating, and Analyzing Survey Replicate Weights*, 2025. URL <https://CRAN.R-project.org/package=svrep>. R package version 0.9.1. [p1, 19]
- J. Shao and R. R. Sitter. Bootstrap for imputed survey data. *Journal of the American Statistical Association*, 91(435):1278–1288, 1996. doi: 10.1080/01621459.1996.10476997. [p2]
- R. R. Sitter. *Resampling Procedures for Complex Survey Data*. PhD thesis, University of Waterloo, Waterloo, ON, Canada, 1989. [p8]
- R. R. Sitter. Comparing three bootstrap methods for survey data. *The Canadian Journal of Statistics*, 20(2):135–154, 1992a. doi: 10.2307/3315464. [p1, 3, 4, 7, 8, 11]
- R. R. Sitter. A resampling procedure for complex survey data. *Journal of the American Statistical Association*, 87(419):755–765, 1992b. doi: 10.1080/01621459.1992.10475277. [p1, 4, 5, 12]

Zeinab Mashreghi
 University of Winnipeg
 Department of Mathematics and Statistics
 Winnipeg, Canada
 ORCID: 0000-0003-0563-0792
z.mashreghi@uwinnipeg.ca